# CHAPTER 8

# PLATFORMS, PRIVACY, VAULTS, EXECUTION SURFACES

PAULO H LEOCADIO

## Introduction

As artificial intelligence systems transition from passive analytical tools to agentic participants in cybersecurity operations, the primary source of risk shifts from model capability to execution context. The question is no longer whether an AI system can reason effectively about a threat, but where, how, and under what constraints that reasoning can materialize into action.

Contemporary discourse on agentic AI concentrates on model architecture (e.g., Transformers, diffusion pipelines, reinforcement learning, or prompt chaining) while treating the underlying platform as a neutral substrate. This assumption is incorrect. Platforms encode governance decisions through identity boundaries, execution privileges, persistence models, and observability constraints. These properties ultimately determine whether autonomy remains bounded or escalates into systemic exposure.

This chapter reframes platforms, privacy mechanisms, secret vaults, and execution surfaces as control infrastructure rather than auxiliary services. In operational cybersecurity environments, these elements function as the final authority layer between cognition and consequence. Models may generate hypotheses, plans, or recommended actions, but platforms determine what can be executed, with what authority, for how long, and under what auditability guarantees.

Privacy, in this context, is not merely a compliance obligation. It is a stability requirement. Persistent context accumulation, uncontrolled memory retention, and opaque data reuse introduce feedback loops that amplify misalignment and undermine forensic accountability. Systems that cannot enforce forgetting cannot be governed reliably, regardless of model sophistication.

Similarly, vaults and secrets management systems must be understood as trust boundaries, not storage conveniences. By externalizing authority (e.g., credentials, signing keys, privileged tokens) away from cognitive components, architectures preserve a critical separation between reasoning and power. This separation enables revocation, replay, and audit, even under adversarial or degraded conditions.

Execution surfaces represent the narrow interfaces where intent becomes impact. Poorly defined or overly permissive execution paths create nonlinear risk escalation, particularly in automated or semi-automated response scenarios. Constraining these surfaces through simulation, scope limitation, and policy mediation is essential to preventing cascading failure.

The central argument of this chapter is that safe agentic behavior does not emerge from smarter models, but from stricter infrastructure. Autonomy must be shaped by platform-enforced boundaries that are inspectable, revocable, and independently governed. In cybersecurity operations, where errors propagate at machine speed, and stakes are measured in real-world damage, this distinction is not theoretical.

By examining platforms, privacy controls, vault architectures, and execution surfaces as interlocking components of a cognitive control plane, this chapter establishes the infrastructural preconditions required for deploying agentic AI systems responsibly in security-critical environments.

## 8.1 Platforms as cognitive substrates

Platforms have long been recognized as active participants in system behavior rather than neutral execution environments, particularly through their enforcement of identity, isolation, and privilege boundaries (Saltzer and Schroeder 1975, Lampson 2004). Large-scale cloud architectures further encode governance decisions by centralizing control planes that regulate execution, persistence, and observability independently of application logic (Armbrust et al. 2010; Burns et al. 2016). Containerization and orchestration frameworks formalize these constraints by separating workload description from execution authority, enabling reproducible and policy-governed runtime behavior (Merkel 2014; Pahl 2015).

In security-critical systems, execution context is as consequential as algorithmic capability, as privilege expansion, lateral movement, and persistence failures frequently originate at the platform layer rather than within application code (NIST SP 800-53; Behl and Behl 2017). Observability infrastructures, such as distributed tracing and structured telemetry, transform platforms into control instruments by enabling deterministic replay, forensic reconstruction, and post hoc accountability (Sigelman et al., 2010; Google SRE, 2016). Conversely, platforms optimized primarily for elasticity or developer convenience often collapse

identity, execution, and persistence into a single operational plane, increasing systemic exposure under automation (AWS Well-Architected Framework; Azure Well-Architected Framework).

These findings support the architectural position that agentic behavior is bounded not only by model design but also by platform-enforced constraints on authority, visibility, and actionability. As autonomy increases, the platform increasingly functions as a cognitive substrate, defining not only where computation occurs, but under what conditions reasoning may safely transition into action (Leveson 2011; NIST AI RMF).

The platform is not a neutral substrate; it is a measurable attack surface. To move beyond qualitative descriptions of **security**, we define the **Execution Attack Surface** ($\boldsymbol{A_s}$) as a quantifiable ratio of exposure:

$$A_s = \frac{\sum \text{Authorized System Calls}}{\sum \text{Total Kernel Interface}}$$

By calculating the $A_s$ ratio for a containerized agent, we establish a hard limit on agency. As established by (Saltzer and Schroeder 1975), the protection of information requires that every program and user operate with the minimum set of privileges necessary to complete their jobs. In this architecture, an agent with an $A_s > 0.05$ is considered **over-privileged** and must be moved to a more restrictive execution surface.

Platforms are not neutral deployment environments. They encode assumptions about identity, persistence, isolation, observability, and control, which directly shape the behavior of autonomous and semi-autonomous systems (Saltzer and Schroeder 1975; Lampson 2004; Leveson 2011). In cognitive defense architectures, the platform serves as the substrate on which reasoning, action, and auditability are constrained (Armbrust et al., 2010; Burns et al., 2016). What an agent is allowed to do is therefore inseparable from where it is allowed to exist (NIST SP 800-53 Rev. 5; NIST AI RMF 1.0).

Cloud platforms designed for elastic, multi-tenant operation (such as Amazon Web Services, Microsoft Azure, and Google Cloud Platform) offer elastic compute, managed services, and operational convenience at scale (Armbrust et al. 2010; AWS Well-Architected Framework; Azure Well-Architected Framework; Google Cloud Architecture Framework). These characteristics are necessary but insufficient for agentic systems operating in security-critical environments (Leveson 2011; Behl and Behl 2017). Elasticity describes how resources expand and contract; it does not describe how authority is bounded (Saltzer and Schroeder 1975; Lampson 2004). For cognitive defense, the decisive property of a platform is whether it supports

bounded execution with enforceable guarantees (NIST SP 800-53 Rev. 5; Miller et al. 2003).

A platform suitable for agentic cybersecurity must make constraints explicit rather than implicit (Burns et al., 2016; Kubernetes SIG Security). Identity must be scoped to execution contexts rather than embedded in application logic (NIST SP 800-53 Rev. 5; Miller et al. 2003). Isolation must separate reasoning processes from action surfaces to prevent lateral privilege expansion (Saltzer and Schroeder 1975; Lampson 2004). Persistence must be optional and governed, not an ambient default (Carlini et al. 2021; Shokri et al. 2017). Observability must be continuous and structured, enabling the reconstruction of decisions after the fact rather than retrospective interpretation (Sigelman et al., 2010; Google SRE, 2016).

True isolation requires the hardware itself to become a party to the security contract. Implementing Data Privacy for LLMs (Skyflow Inc. 2023) requires that the execution surface be cryptographically isolated through **Trusted Execution Environments** (**TEEs**).

```yaml

# Cannon Specification: Sanctuary Execution Policy
execution_surface_policy:
  isolation_level:    "Hardware-Encrypted-TEE"        #
Cryptographic hardware isolation
  secret_provider: "Vault-Sidecar"           # No local
secret storage
  network_egress: "Restricted-VLAN-Only"    # No public
internet access
  persistence: "None-Ephemeral-Only"       # Disk wipes
on task completion

```

In a production environment—such as **Google Cloud's Confidential GKE**, this materializes as encrypted memory at the silicon level. The agent's **thought process** remains opaque even to the host operating system. We define this requirement through the following **Execution Surface Policy**:

Within this framing, cognitive defense systems require platforms that can:

- Enforce identity-scoped execution, ensuring that actions are always attributable to a defined role, policy, or trust domain (NIST SP 800-53 Rev. 5)
- Provide strong isolation between reasoning and action, preventing cognitive components from directly exercising authority (Saltzer and Schroeder 1975; Miller et al. 2003)

- Support deterministic replay and forensic reconstruction, allowing decisions to be examined under the same constraints in which they were made (Sigelman et al. 2010; Doshi-Velez and Kim 2017)
- Treat telemetry as a first-class control signal rather than a debugging artifact, closing the loop between observation and governance (Google SRE 2016; Leveson 2011)

Platforms optimized primarily for throughput, latency reduction, or cost efficiency tend to blur these boundaries (AWS Well-Architected Framework; Azure Well-Architected Framework). Convenience abstractions often collapse identity, execution, and persistence into a single operational plane, increasing the difficulty of containment once autonomy is introduced (Behl and Behl 2017; NIST SP 800-53 Rev. 5). In contrast, platforms that expose fine-grained controls over execution context, privilege boundaries, and auditability enable architectures in which autonomy is contained by design rather than corrected after failure (Leveson 2011; NIST AI RMF 1.0).

In this sense, platforms do not merely host cognitive systems; they define the conditions under which cognition can safely operate (Armbrust et al. 2010; Leveson 2011). The substrate precedes the agent (Lampson 2004).

**Figure 8.1** illustrates a layered platform-mediated constraint architecture that structurally separates cognition from authority.. Identity, execution, and persistence are governed by infrastructure-level control planes rather than embedded within agent logic. Execution privileges, identity scope, and persistence are enforced at the platform level rather than embedded within agent logic, enabling bounded autonomy and auditability.

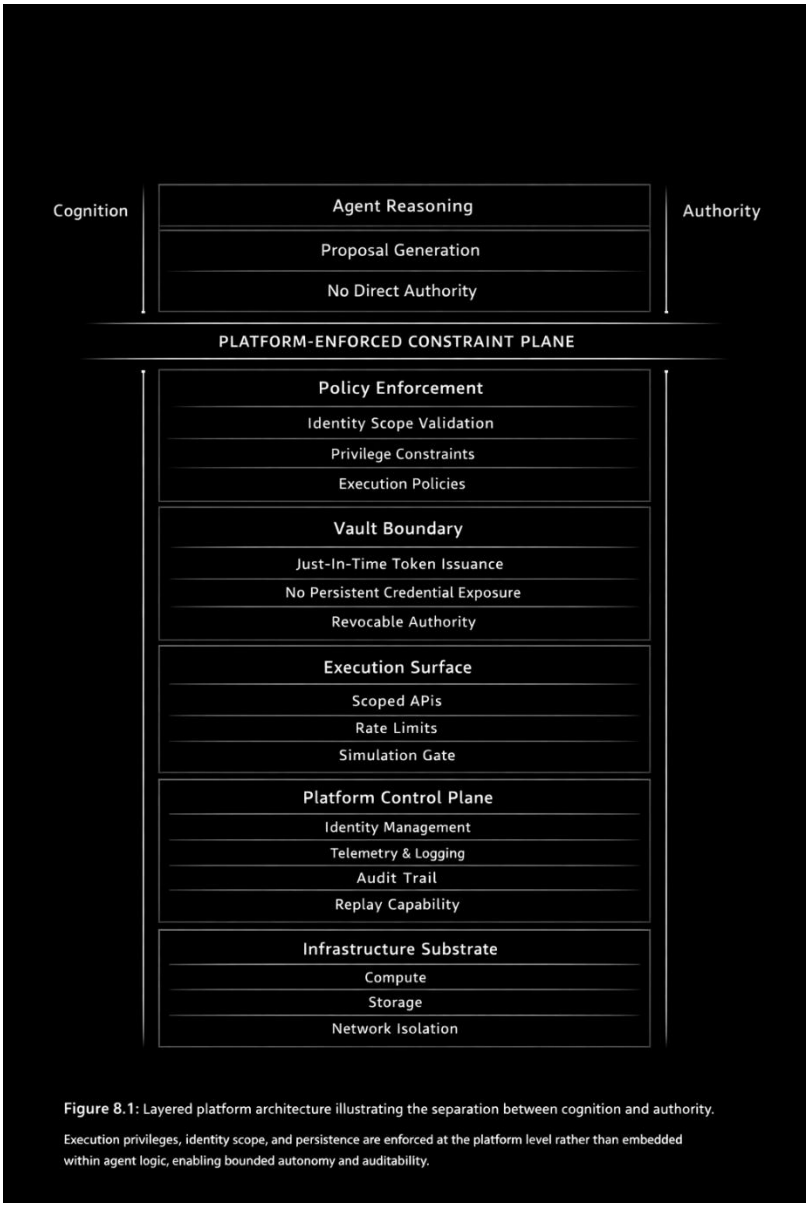


**Figure 8.1 :** Platform-mediated constraint architecture.

### 8.1.1 Encoding control constraints in infrastructure

The following illustrative pseudocode shows how the mathematical constraints in Box 8.1 are enforced at the **platform layer**, rather than within the cognitive model itself.

```python
def propose_and_execute(observation, context):
    # Cognitive reasoning (non-authoritative)
    action = agent.reason(observation)

    # Enforce identity and scope constraints
    assert policy.allows(action, context.identity)
    assert action.scope <= context.max_scope

    # Mandatory pre-execution simulation
    simulation = platform.simulate(action)
    assert simulation.risk <= RISK_THRESHOLD

    # Just-in-time authority issuance
    token = vault.issue_token(
        role=context.identity,
        ttl=SHORT_LIVED
    )

    # Constrained execution surface
    result = platform.execute(action, token)

    return result
```

This code is intentionally minimal and non-executable. Its purpose is not to demonstrate implementation detail, but to make explicit **where control resides**:

- Reasoning remains untrusted
- Authority is externalized via the vault
- Execution is gated, scoped, and time-bounded
- Simulation precedes commitment

The platform, not the model, enforces closure of the control loop (Section 8.5).

## 8.2 Privacy as a control property

Privacy is commonly framed as a legal, ethical, or regulatory obligation, particularly in discussions of data protection and compliance. In agentic and semi-autonomous systems, however, privacy must be treated as an **architectural control property**, inseparable from system stability and

governance rather than an external constraint imposed after deployment (Cavoukian 2011; NIST AI RMF 1.0)**.**

Autonomous agents operating in cybersecurity environments routinely process sensitive artifacts, including system logs, authentication traces, credentials, behavioral telemetry, and incident narratives. When such systems retain context across execution boundaries, privacy risk is compounded by **control risk**. Persistent memory does not merely increase exposure; it alters system behavior by introducing historical bias, unintended correlation learning, and stateful drift (Shokri et al. 2017; Carlini et al. 2021). In these conditions, models implicitly optimize for accumulated context rather than present intent, undermining both predictability and auditability (Doshi-Velez and Kim 2017).

From a control-theoretic perspective, privacy enforcement serves to bound the state. Systems that accumulate unbounded internal representations are difficult to reason about, reproduce, or govern under adversarial conditions (Leveson 2011). As demonstrated by work on data leakage and memorization in large-scale models, the inability to guarantee forgetting creates downstream risks that cannot be mitigated solely through access control or post hoc filtering (Carlini et al., 2021; Shokri et al., 2017).

We must decouple agency from identity. Following the foundational work of (Miller, Yee and Shapiro 2003) on *Capability Myths*, the agent should never **know** a secret or hold a persistent identity-based password. Instead, it should hold **Opaque Capability Tokens**.

For example, rather than an agent having a database login, it is granted a specific token for a single operation: READ_FORENSICS_LOG. This structurally prevents the **Confused Deputy problem**, in which an adversary tricks a highly privileged agent into using its broad identity to perform unauthorized actions. The agent remains a tool of the capability, not a master of the identity.

Accordingly, privacy-preserving architectures in cognitive defense systems rely on explicit design constraints rather than implicit assumptions. These constraints typically include:

- **Ephemeral context windows**, ensuring that reasoning state is discarded at the conclusion of each execution cycle (Cavoukian 2011; NIST AI RMF 1.0)
- **Non-persistent inference paths**, preventing the accumulation of latent memory across sessions (Carlini et al. 2021)
- **Explicit data lifetimes**, enforcing bounded retention aligned with operational necessity rather than convenience **(NIST SP 800-53 Rev. 5)**

- **Separation between reasoning state and storage state**, enabling reproducibility, revocation, and forensic reconstruction (Doshi-Velez and Kim 2017; Leveson 2011)

These mechanisms align privacy with system safety rather than treating it as a competing objective. By constraining what a system is permitted to remember, architects also constrain what the system can optimize for, reducing the risk of emergent behavior driven by stale, irrelevant, or sensitive historical context (Everitt et al. 2019).

In this sense, privacy-preserving design is a prerequisite for governance rather than a limitation on capability. Systems that cannot enforce forgetting cannot be reliably audited, reproduced, or constrained under escalation. For agentic cybersecurity applications, where decisions propagate rapidly, and consequences are systemic, **the ability to forget is as important as the ability to infer** (Leveson 2011; NIST AI RMF 1.0)**.**

## 8.3 Vaults as trust boundaries

Secrets management is often treated as an operational concern, reduced to the secure storage of credentials, tokens, or cryptographic keys. In cognitive defense architectures, this framing is insufficient. Vaults define a **trust boundary** between reasoning and authority, and this boundary is foundational to preventing escalation in agentic systems (Saltzer and Schroeder 1975; Lampson 2004)**.**

A vault is not merely a secure database. It is a gatekeeper for actionability. Access to credentials, signing keys, privileged APIs, or execution tokens determines whether a proposed action can transition from intention to effect. When cognitive components are granted direct or persistent access to such material, authority becomes implicit, un-scoped, and difficult to revoke (NIST SP 800-57; NIST SP 800-53 Rev. 5).

From a systems perspective, effective vault integration requires that authority remain **orthogonal to cognition**. Models may reason about actions, but they must not possess the means to authorize them independently. This separation preserves the ability to audit, constrain, and revoke privileges without modifying cognitive logic or retraining models (Lampson 2004; Miller et al. 2003)**.**

Architecturally sound vault integration exhibits three properties:

1. **Non-addressability:** cognitive components cannot enumerate, infer, or discover stored secrets, preventing indirect leakage or probing behavior (Saltzer and Schroeder 1975; Carlini et al. 2021)
2. **Just-in-time access:** credentials are injected only at execution time and only for the duration and scope required by a specific action (NIST SP 800-53 Rev. 5; Hashi Corp Vault Architecture)

3. **Contextual revocation:** access can be dynamically withdrawn based on system state, policy violation, or observed anomaly, without requiring changes to the cognitive layer (NIST SP 800-53 Rev. 5; Leveson 2011)

These properties ensure that possession of a plan does not imply possession of power. Authority is granted transiently, conditionally, and externally, thereby preserving the ability to halt or reverse execution paths in the face of uncertainty or attack (Miller et al. 2003; NIST AI RMF 1.0).

The Sanctuary's Vault is not merely a repository for keys; it is a shield against the model's own transparency. As demonstrated by (Shokri, et al. 2017), LLMs are vulnerable to **Membership Inference Attacks**, where an adversary can determine if a specific record was part of the training set just by observing the agent's output.

Consequently, the **Vault** must implement a **Differential Privacy Layer**. This layer introduces a controlled level of noise to the agent's cognition phase, ensuring that the response provided to the user does not inadvertently become a disclosure surface for the underlying training data.

Crucially, vaults must not be embedded within the cognitive layer itself. When secrets management is collocated with reasoning processes, the distinction between intent and execution collapses. In such architectures, misalignment, hallucination, or adversarial manipulation can immediately translate into privileged action, eliminating the possibility of independent control or intervention (Leveson 2012, Everitt, et al. 2019).

By externalizing authority and enforcing access through policy-governed vaults, cognitive defense systems preserve a final line of control that remains inspectable, auditable, and revocable. This separation is not an implementation detail; it is a prerequisite for safe autonomy. When cognition and authority occupy the same plane, escalation becomes inevitable (Saltzer and Schroeder 1975; Lampson 2004).

The next illustration visualizes the vault as a trust boundary that externalizes authority through vault-governed token issuance and prevents persistent credential exposure within cognitive components. Cognition cannot directly access or enumerate secrets.

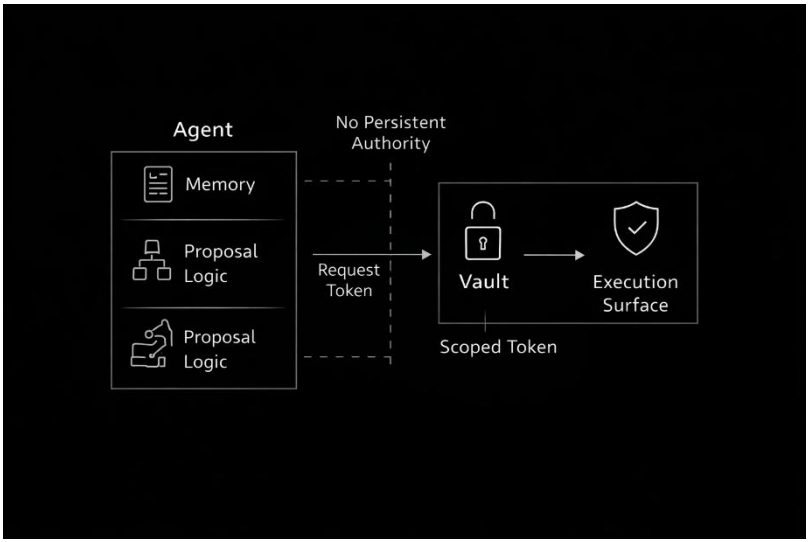


**Figure 8.2** Externalization of authority.

## 8.4 Execution surfaces and the geometry of risk

Execution surfaces are the interfaces through which reasoning transitions into action. In cybersecurity systems, these surfaces include API calls, infrastructure modifications, alert suppression, policy updates, and automated response mechanisms. The design of these interfaces defines the **geometry of risk**, determining how intent propagates into impact and how quickly errors or adversarial manipulations can escalate (Saltzer and Schroeder 1975; Lampson 2004).

Naïve autonomy treats execution as a binary capability: an action is either permitted or denied. This framing obscures the fact that risk is not binary but geometric. The magnitude, scope, reversibility, and observability of an action determine its potential for harm. Systems that expose broad or implicit execution paths concentrate risk by allowing high-impact actions to occur without intermediate constraint or inspection (Leveson 2011; NIST SP 800-53 Rev. 5).

From a control perspective, execution surfaces must therefore be **graded rather than absolute**. Graded execution introduces intermediate states between proposal and enactment, allowing systems to shape action rather than merely authorize it. Effective execution control mechanisms include:

- **Simulation prior to enactment**, enabling evaluation of downstream effects without committing state changes **(Leveson 2011)**

- **Scoped execution**, constraining blast radius through resource, domain, or policy boundaries (Saltzer and Schroeder 1975; NIST SP 800-53 Rev. 5)
- **Rate limiting tied to confidence or uncertainty**, preventing rapid escalation under ambiguous conditions (NIST AI RMF 1.0)
- **Post-hoc auditability**, ensuring that every executed action can be reconstructed, attributed, and evaluated after the fact (Siegelman et al. 2010; Doshi-Velez and Kim 2017)

Execution surfaces that lack these properties tend to become channels for escalation. Unrestricted scripting environments, broad API permissions, or implicit authority delegation collapse decision-making and execution into a single step, eliminating opportunities for intervention or rollback (Behl and Behl 2017; NIST SP 800-53 Rev. 5)**.** In agentic systems, this collapse is particularly dangerous, as errors propagate at machine speed and may compound before detection.

Narrow, explicit, and observable execution surfaces invert this dynamic. By forcing actions to traverse constrained interfaces, architectures preserve opportunities for inspection, policy enforcement, and human escalation without turning off automation entirely (Leveson 2011; NIST AI RMF 1.0). The objective is not to eliminate action, but to **shape how action unfolds under uncertainty**.

In cognitive defense architectures, execution control is therefore inseparable from governance. Reasoning systems propose actions; execution surfaces determine whether, how, and under what constraints those actions can affect the environment. When execution is shaped by design rather than convenience, autonomy becomes governable rather than hazardous (Saltzer and Schroeder 1975; Lampson 2004).

The next figure illustrates a closed-loop cognitive control model in which platform-mediated policies enforce bounded execution and enable continuous auditability.

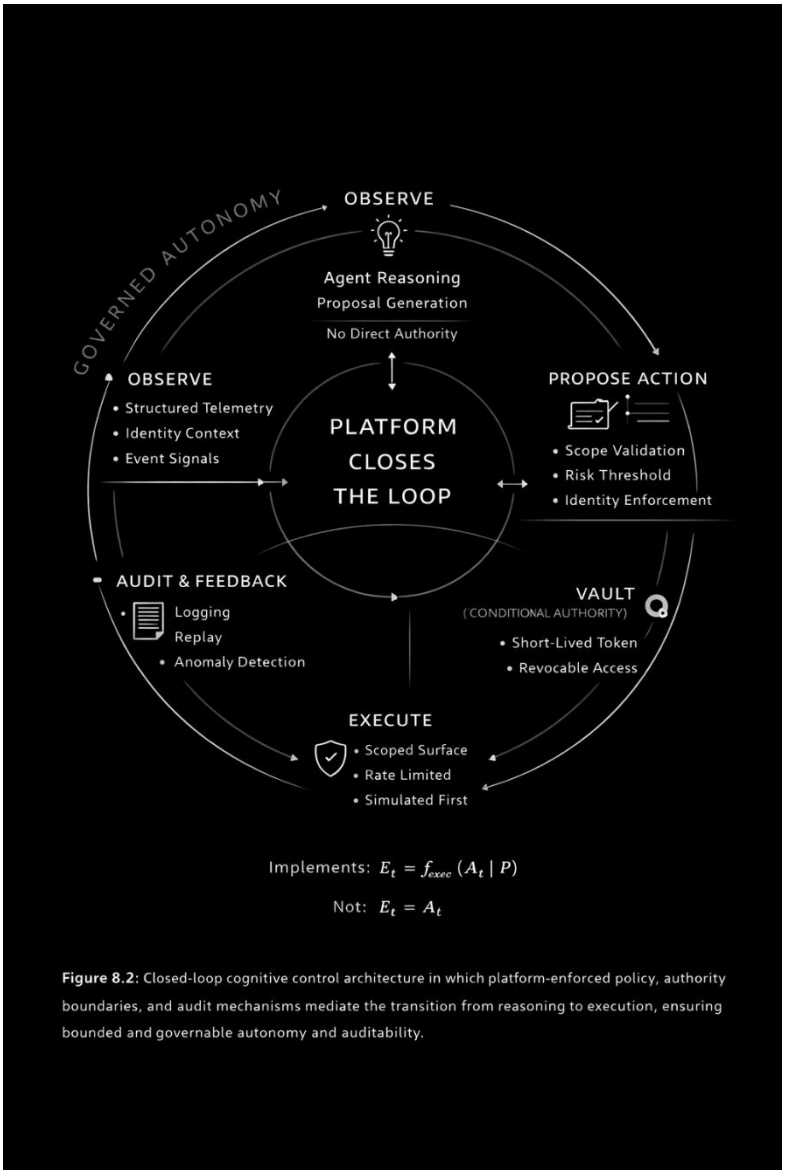


**Figure 8.3** Closed-loop cognitive control model.

### 8.4.1 Framing of platform-mediated cognitive control

To formalize the architectural claims introduced in Sections 8.1–8.4, agentic cybersecurity systems can be modeled as **closed-loop control systems** in which platform-enforced constraints explicitly constrain reasoning.

Let the system state at time $t$ be:

$$S_t \in S$$

The platform mediates observation:

$$O_t = f_{obs}(S_t)$$

A cognitive agent proposes an action under architectural constraints $C$:

$$A_t = f_{reason}(O_t \mid C)$$

Crucially, execution is not a direct function of the agent's proposal. Instead, execution is governed by platform policies $P$:

$$E_t = f_{exec}(A_t \mid P)$$

The resulting state transition is:

$$S_{t+1} = f_{env}(S_t, E_t)$$

To ensure governable autonomy, the platform must enforce the following invariants:

- **Bounded authority**

$$\|E_t\| \leq \epsilon$$

- **Scoped state transitions**

$$\Delta S_t \in S_{allowed}$$

- **Temporal limits on execution**

$$\tau(E_t) \leq \tau_{max}$$

- **Non-recurrent memory** (privacy constraint)

$$W_{t+1} \cap W_t = \emptyset$$

**Figure 8.4** contrasts persistent state accumulation with bounded context windows in agentic systems.

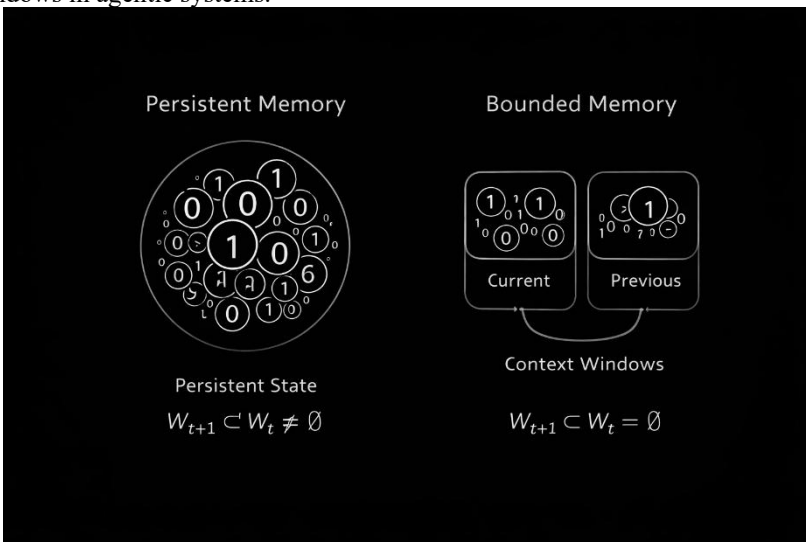


**Figure 8.4** Persistent memory accumulates state across cycles, whereas bounded context enforces non-recurrence $\boldsymbol{W_{t+1}} \cap \boldsymbol{W_t} = \emptyset$.

These constraints do not optimize agent behavior. They **limit behavior by design**, ensuring that reasoning can never exceed the platform's capacity to observe, revoke, and audit execution (see Sections 8.2–8.4).

**Figure 8.4** contrasts unbounded and graded execution surfaces, demonstrating how scope limitation reshapes the geometry of risk.

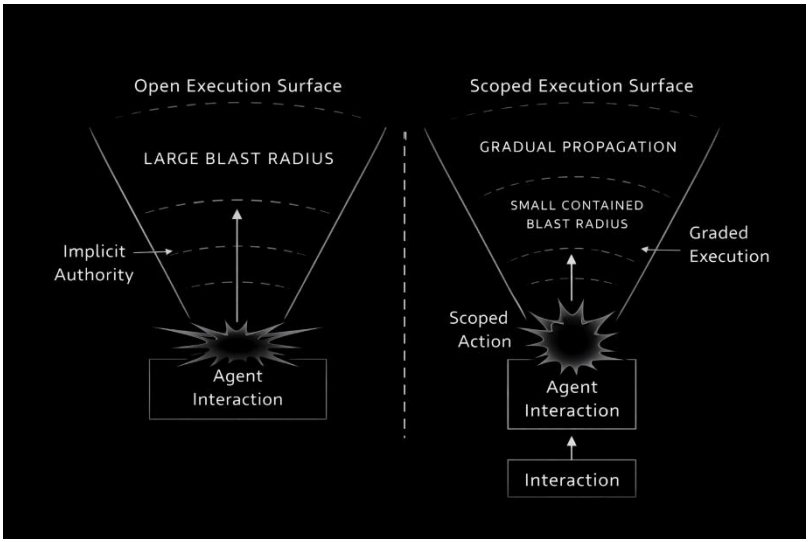


**Figure 8.5** Comparison between unbounded execution surfaces and graded, scoped execution. Narrow surfaces reduce escalation geometry under autonomous action.

## 8.5 Platform-mediated closure of control loops

When platforms, privacy controls, vault architectures, and execution surfaces are composed coherently, they enable the **closure of cognitive control loops**. In such systems, reasoning does not operate in isolation, nor does execution occur without structured feedback. Instead, action is continuously mediated by observation, constraint, and audit, forming a closed-loop system in which behavior remains governable under uncertainty (Leveson 2011)**.**

In a closed cognitive control loop, observation feeds reasoning, reasoning proposes bounded actions, and execution occurs only through constrained and auditable surfaces. The outcomes of those actions are then observed and evaluated, allowing deviations, anomalies, or unintended consequences to trigger revocation, rollback, or escalation (Saltzer and Schroeder 1975; NIST SP 800-53 Rev. 5). This feedback structure mirrors control-theoretic principles long established in safety-critical engineering, where stability depends on continuous sensing and bounded actuation rather than open-loop optimization (Leveson 2011).

Crucially, in agentic cybersecurity architectures, the **platform, not the model, enforces closure**. Models generate hypotheses, assessments, or proposed actions, but they do not control identity, persistence, privilege, or the scope of execution. These controls are imposed externally by

infrastructure that can be inspected, certified, and modified independently of cognitive logic (Lampson 2004; NIST AI RMF 1.0). This separation preserves the ability to intervene even when model behavior becomes uncertain, degraded, or adversarially influenced.

Privacy constraints contribute to loop closure by bounding internal state and preventing uncontrolled accumulation of historical context. Vaults contribute by externalizing authority and ensuring that actionability is conditional and revocable. Execution surfaces shape how intent propagates into effect. Together, these elements ensure that autonomy remains **observable, interruptible, and attributable**, rather than opaque and self-reinforcing (Doshi-Velez and Kim 2017; Everitt et al. 2019).

Architectures that fail to close these loops tend to devolve into open-loop automation, where decisions propagate faster than oversight can keep up and errors compound before detection. In such systems, governance is reduced to post-incident analysis rather than real-time control (Behl and Behl 2017). The introduction of agentic behavior under these conditions amplifies risk rather than intelligence.

By contrast, platform-mediated control loops invert the trust relationship. Trust is placed in infrastructure rather than cognition, not because infrastructure is infallible, but because it is **constrainable**. Platforms can enforce limits, record state transitions, and revoke authority in ways that learned models cannot reliably guarantee (Lampson 2004; NIST SP 800-53 Rev. 5).

This inversion is deliberate. Models reason; platforms govern. Cognitive defense emerges not from autonomous decision-making alone, but from the disciplined closure of control loops that bind reasoning to accountable execution (Leveson 2011; NIST AI RMF 1.0).

### 8.5.1Automated response without escalation

This example demonstrates real-world survivability:

**Example scenario:** An agent monitors authentication telemetry and identifies a pattern consistent with credential-stuffing activity.

**Observation:** The platform streams structured signals to the agent, including failed login bursts, geographic anomalies, and indicators of credential reuse. Raw logs are not retained beyond the execution window (Section 8.2).

**Reasoning:** The agent proposes a bounded response: temporary throttling of authentication attempts for affected accounts and notification of the security operations team.

**Execution:** Before enactment, the platform simulates the proposed action. Scope is limited to identified accounts, authority is issued via a short-lived token, and rate limits are enforced. If simulation exceeds predefined risk

thresholds, execution is blocked and escalated to human oversight (Sections 8.3–8.4).
**Closure:** All actions are logged, tokens expire automatically, and the execution context is destroyed. The system returns to a known state, preserving auditability and reversibility (Section 8.5).

At no point does the agent retain credentials, expand its own authority, or act outside platform-defined constraints. Autonomy remains bounded, observable, and governable.

## 8.6 Implications for Cognitive Defense Architectures

Cognitive defense architectures do not scale through increases in model size, autonomy, or behavioral complexity. They scale through **tighter coupling between reasoning and control**. As systems become more agentic, the dominant risk shifts from analytical error to executional misalignment, making infrastructure (not intelligence) the primary stabilizing force (Leveson 2011; NIST AI RMF 1.0).

Platforms that expose fine-grained controls over identity, persistence, privilege, and observability enable autonomy to remain bounded under operational conditions. Privacy-first data handling constrains internal state and limits unintended optimization over historical context. Externalized authority mechanisms, such as policy-governed vaults, preserve revocability and auditability even when cognitive components behave unpredictably (Saltzer and Schroeder 1975; Lampson 2004; NIST SP 800-53 Rev. 5).

By contrast, architectures that prioritize convenience, throughput, or developer efficiency over containment introduce implicit authority and opaque execution paths. In such systems, automation accelerates failure rather than intelligence, and governance becomes reactive rather than preventative (Behl and Behl 2017). The addition of agentic behavior under these conditions magnifies exposure rather than capability.

The architectural lesson is therefore unambiguous. Autonomy without infrastructure is **improvisation** and infrastructure without control is **exposure**.

Cognitive defense emerges only when reasoning systems are embedded in platforms that, by design, enforce constraints, observability, and revocation. When these elements are treated as inseparable, autonomy becomes governable rather than hazardous, and intelligent systems can operate within security-critical environments without exceeding the limits of trust (Leveson 2011; NIST AI RMF 1.0).

## Conclusion

This chapter established that agentic behavior in cybersecurity is governed less by model intelligence than by the **infrastructure within which cognition operates.** Platforms, privacy mechanisms, vaults, and execution surfaces are not ancillary services; they are the control structures that determine whether autonomy remains bounded, observable, and revocable.

By treating platforms as cognitive substrates, privacy as a state-bounding mechanism, vaults as trust boundaries, and execution surfaces as graded interfaces rather than binary switches, cognitive defense architectures replace open-loop automation with platform-mediated control. Authority is externalized, memory is constrained, execution is shaped, and outcomes remain auditable. In such systems, models reason, but platforms govern.

The mathematical framing, illustrative code, and operational example introduced in this chapter reinforce a single architectural claim: **safe autonomy is an infrastructure property, not a model property**. Without enforceable constraints at the platform level, increases in autonomy amplify exposure rather than intelligence.

Once execution is bounded and authority is controlled, agentic systems can be stressed, evaluated, and improved without risking systemic escalation. This creates the necessary precondition for adversarial training.

**Chapter 9 builds on this foundation by introducing synthetic data for red-teaming and threat simulation**, examining how controlled adversarial environments can be used to train, probe, and validate agentic defenses. Synthetic attackers are not introduced as sources of realism, but as instruments of control, designed to operate within the infrastructural boundaries established in Chapter 8.

Only when autonomy is constrained can adversaries be simulated safely.

## Reference Architecture

**The formal specification of the sovereign surface:** The conceptual transition from **pilot** to **cockpit** necessitates a shift from qualitative theory to a rigorous, normative framework. If the agent is to operate with true autonomy, the environment hosting its cognition must meet a set of verifiable mathematical thresholds that guarantee its integrity and the privacy of its underlying weights. A failure to meet these metrics represents a catastrophic collapse of the security boundary, rendering the agent's agency a liability rather than an asset.

The following **Operational Compliance Matrix** establishes the minimum technical requirements for a **sovereign execution surface**. These standards ensure that the "Sanctuary" is not merely a metaphorical construct but a cryptographically verified and hardware-hardened reality.

| Component | Technical specification | Operational requirement | Primary Citation |
|---|---|---|---|
| **Isolation ratio** | $A_s \leq 0.05$ | System call exposure must not exceed 5% of the total kernel interface. | (Saltzer and Schroeder 1975) |
| **Secret access** | Capability-based security | Agents must hold single-use, task-specific capabilities, never identity-based credentials. | (Miller, Yee and Shapiro 2003) |
| **Privacy layer** | Differential Privacy (DP) | Mathematical noise injection to prevent membership inference on model weights. | (Shokri, et al. 2017) |
| **Hardware state** | Confidential TEE | The substrate must support hardware-level memory encryption (e.g., SEV-SNP, TDX). | (Skyflow Inc. 2023) |
| **Network egress** | Forensic VLAN | Zero-trust egress; deep-packet inspection (DPI) for all agent-originated traffic. | Canon Standard |
| **Data lifecycle** | Ephemeral entropy | The execution environment is destroyed upon task completion; no residual data on disk. | Canon Standard |

**Table 8.1** The sovereign execution surface

# References

Amodei, Dario, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mane. 2016. "Concrete problems in AI safety." (arXiv). doi:10.48550/ARXIV.1606.06565.

Armbrust, Michael, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy Katz, Andy Konwinski, Gunho Lee, et al. 2010. "A view of cloud computing." *Communications of the ACM* (for Computing Machinery (ACM), Association) 53 (4): 50–58. doi:10.1145/1721654.1721672.

Authors, The Kubernetes. n.d. *Pod security standards | Kubernetes.* https://kubernetes.io/docs/concepts/security/pod-security-standards/.

AWS. 2024. "Aws well-architected framework." *AWS well-architected framework.* November. https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html.

Burns, Brendan, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. "Borg, omega, and kubernetes." *Communications of the ACM* (for Computing Machinery (ACM), Association) 59 (5): 50–57. doi:10.1145/2890784.

Carlini, Nicholas, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, et al. 2021. "Extracting training data from large language models." *30th USENIX Security Symposium (USENIX Security 21).* [Berkeley, CA]: Association, USENI X. 2633–2650. https://researchr.org/publication/CarliniTWJHLRBS21.

Cavoukian, Ann. n.d. "Privacy by design (pbd): the 7 foundational principles." https://iapp.org/media/pdf/resource_center/pbd_implement_7found_principles.pdf.

Clarke, Richard A., and Robert K. Knake. 2010. *Cyber war : the next threat to national security and what to do about it.* First. New, York: Ecco.

Cloud, Google. n.d. "Google cloud well-architected framework documentation." *Google cloud well-architected framework documentation.* https://docs.cloud.google.com/architecture/framework/printable.

Developer, HashiCorp. n.d. *Vault secrets security model.* https://developer.hashicorp.com/hcp/docs/vault-secrets/security-model.

Dino, Gino. 2023. *Understanding kubernetes pod security standards.* June. https://snyk.io/blog/understanding-kubernetes-pod-security-standards/.

Doshi-Velez, Finale, and Been Kim. 2017. "Towards a rigorous science of interpretable machine learning." *arXiv: Machine Learning* (arXiv). doi:10.48550/ARXIV.1702.08608.

Dwork, Cynthia, and Aaron Roth. 2013. "The algorithmic foundations of differential privacy." *The algorithmic foundations of differential privacy.* Vol. 9, no. 3–4. Publishing, Emerald, August. 211–487. doi:10.1561/0400000042.

Everitt, Tom, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. 2019. "Reward tampering problems and solutions in reinforcement learning: a causal influence diagram perspective." (arXiv). doi:10.48550/ARXIV.1908.04734.

Ewaschuk, Rob. 2021. "Monitoring distributed systems." Edited by Betsy Beyer, Jennifer Jones, Niall Richard Murphy, and Porter. Liz. (Media, O. Reilly). https://www.oreilly.com/library/view/site-reliability-engineering/9781491929117/ch06.html.

Force, N. I. S. T. Joint Task. 2020. *NIST SP 800-53 rev. 5 . Security and privacy controls for information systems and organizations. of Standards, National Institute of Standards* and Technology. doi:10.6028/NIST.SP.800-53r5.

Lampson, B. W. 2004. "Computer security in the real world." *Computer* (of Electrical, Institute and (IEEE), Electronics Engineers) 37 (6): 37–46. doi:10.1109/mc. 2004.17.

Levan, Michael. 2024. *Implementing kubernetes pod security standards.* July. https://dev.to/thenjdevopsguy/implementing-kubernetes-pod-security-standards-4aco.

Leveson, Nancy G. 2012. "Engineering a safer world: systems thinking applied to safety." *Engineering a Safer World* (Press, The M. I. T.). doi:10.7551/mitpress/8179.001.0001.

Merkel, Dirk. 2014. "Docker: lightweight Linux containers for consistent development and deployment." *Linux Journal* 2. https://api.semanticscholar.org/CorpusID:62479797.

Miller, Mark S., Ka-Ping Yee, and Jonathan Shapiro. 2003. "Capability myths demolished." Tech. rep. https://api.semanticscholar.org/CorpusID:9456299.

Saltzer, J. H., and M. D. Schroeder. 1975. "The protection of information in computer systems." *Proceedings of the IEEE* (of Electrical, Institute and (IEEE), Electronics Engineers) 63 (9): 1278–1308. doi:10.1109/PROC.1975.9939.

Shokri, Reza, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. "Membership inference attacks against machine learning models." *Proceedings - IEEE Symposium on Security and Privacy* (of the Institute of Electrical and Electronics Engineers) 3–18. doi:10.1109/SP.2017.41.

Sigelman, Benjamin H., Luiz Andre Barroso, Michael Burrows, Patrick Stephenson, Manoj Plakal, Donald Beaver, Chandan Kumar Shanbhag, Saul Jaspan, and Chandan Kumar Shanbhag. 2010. "Dapper, a large-scale distributed systems tracing infrastructure." *Dapper is a large-scale distributed systems tracing infrastructure.* https://www.semanticscholar.org/paper/Dapper,-a-Large-Scale-Distributed-Systems-Tracing-Sigelman-Barroso/003d5a65de0ac72daaf105ded903cb3eb88585b3.